

Lab 9 — Source Code Report

InvoiceDoc2: Discount & Configuration Features

Generated: 5/6/2026, 11:22:23 AM

Files Modified / Created in Lab 9:

1.	[NEW]	Step 1	database/sql/004_invoice_lab3_delta.sql
2.	[UPDATED]	Step 2	server/src/services/salesPersons.service.js
3.	[UPDATED]	Step 2	server/src/controllers/salesPersons.controller.js
4.	[UPDATED]	Step 2	server/src/routes/salesPersons.routes.js
5.	[NEW]	Step 3	server/src/services/configuration.service.js
6.	[NEW]	Step 3	server/src/controllers/configuration.controller.js
7.	[NEW]	Step 3	server/src/routes/configuration.routes.js
8.	[UPDATED]	Step 3	server/src/app.js
9.	[UPDATED]	Step 4	server/src/services/invoices.service.js
10.	[NEW]	Step 5	client/src/api/configuration.api.js
11.	[UPDATED]	Step 6	client/src/api/salesPersons.api.js
12.	[NEW]	Step 7	client/src/pages/salesPersons/SalesPersonList.jsx
13.	[NEW]	Step 7	client/src/pages/salesPersons/SalesPersonPage.jsx
14.	[UPDATED]	Step 7	client/src/main.jsx
15.	[UPDATED]	Step 8	client/src/components/LineItemsEditor.jsx
16.	[UPDATED]	Step 8	client/src/components/LineItemRow.jsx
17.	[UPDATED]	Step 9	client/src/components/InvoiceForm.jsx
18.	[UPDATED]	Step 10	client/src/pages/invoices/InvoicePage.jsx

DB Delta Script

```
1 \set ON_ERROR_STOP on
2
3 -- Add discount columns to invoice_line_item
4 ALTER TABLE invoice_line_item
5     ADD COLUMN IF NOT EXISTS line_discount_percent numeric(5,2) NOT NULL DEFAULT 0.00,
6     ADD COLUMN IF NOT EXISTS line_discount_amount numeric(14,2) NOT NULL DEFAULT 0.00,
7     ADD COLUMN IF NOT EXISTS line_net_price numeric(14,2) NOT NULL DEFAULT 0.00;
8
9 -- Backfill: set line_net_price = extended_price where no discount
10 UPDATE invoice_line_item
11     SET line_net_price = extended_price
12     WHERE line_discount_amount = 0;
13
14 -- Add totals columns to invoice
15 ALTER TABLE invoice
16     ADD COLUMN IF NOT EXISTS vat_rate numeric(5,4) NOT NULL DEFAULT 0.07,
17     ADD COLUMN IF NOT EXISTS total_price numeric(14,2),
18     ADD COLUMN IF NOT EXISTS total_discount numeric(14,2) NOT NULL DEFAULT 0.00,
19     ADD COLUMN IF NOT EXISTS vat_amount numeric(14,2);
20
21 -- Backfill: total_price = total_amount, vat_amount = vat where NULL
22 UPDATE invoice SET total_price = total_amount WHERE total_price IS NULL;
23 UPDATE invoice SET vat_amount = vat WHERE vat_amount IS NULL;
24
25 -- Create configuration table
26 CREATE TABLE IF NOT EXISTS configuration (
27     id bigint PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,
28     key text UNIQUE NOT NULL,
29     value text NOT NULL
30 );
31
32 -- Seed vat_percent
33 INSERT INTO configuration (key, value) VALUES ('vat_percent', '7') ON CONFLICT DO NOTHING;
34
```

Sales Person Service (CRUD)

```
1 import { pool } from "../db/pool.js";
2
3 export async function listSalesPersons({ search, page, limit }) {
4   const offset = (Number(page) - 1) * Number(limit);
5   const searchParam = `%${search}%`;
6
7   const countResult = await pool.query(
8     `SELECT COUNT(*) as total FROM sales_person
9     WHERE code ILIKE $1 OR name ILIKE $1`,
10    [searchParam],
11  );
12
13  const { rows } = await pool.query(
14    `SELECT id, code, name, start_work_date
15    FROM sales_person
16    WHERE code ILIKE $1 OR name ILIKE $1
17    ORDER BY code ASC
18    LIMIT $2 OFFSET $3`,
19    [searchParam, Number(limit), offset],
20  );
21
22  const total = Number(countResult.rows[0].total);
23  return {
24    data: rows,
25    total,
26    page: Number(page),
27    limit: Number(limit),
28    totalPages: Math.ceil(total / Number(limit)),
29  };
30 }
31
32 export async function getSalesPerson(code) {
33   const { rows } = await pool.query(
34     "SELECT id, code, name, start_work_date FROM sales_person WHERE code = $1",
35     [code],
36   );
37   return rows[0] || null;
38 }
39
40 export async function createSalesPerson({ code, name, start_work_date }) {
41   let resolvedCode = code;
42   if (!resolvedCode || String(resolvedCode).trim() === "") {
43     const maxRes = await pool.query("SELECT MAX(id) as m FROM sales_person");
44     const nextId = (Number(maxRes.rows[0].m) || 0) + 1;
45     resolvedCode = `SP${nextId.toString().padStart(3, "0")}`;
46   }
47   const { rows } = await pool.query(
48     `INSERT INTO sales_person (id, created_at, code, name, start_work_date)
49     VALUES ((SELECT coalesce(max(id),0)+1 FROM sales_person), now(), $1, $2, $3)
50     RETURNING id, code, name, start_work_date`,
51     [resolvedCode, name, start_work_date || null],
52   );
53   return rows[0];
54 }
55
56 export async function updateSalesPerson(code, { name, start_work_date }) {
57   const { rows } = await pool.query(
58     `UPDATE sales_person SET name=$1, start_work_date=$2 WHERE code=$3
59     RETURNING id, code, name, start_work_date`,
60     [name, start_work_date || null, code],
61   );
62   return rows[0] || null;
63 }
64
65 export async function deleteSalesPerson(code) {
66   await pool.query("DELETE FROM sales_person WHERE code=$1", [code]);
67   return { ok: true };
68 }
69
```

Sales Person Controller

```
1 import { listSalesPersons, getSalesPerson, createSalesPerson, updateSalesPerson, deleteSalesPerson } from
  "../services/salesPersons.service.js";
2
3 export async function handleList(req, res) {
4   try {
5     const { search = "", page = 1, limit = 10 } = req.query;
6     const result = await listSalesPersons({ search, page, limit });
7     res.json({
8       success: true,
9       data: result.data,
10      meta: {
11        total: result.total,
12        page: result.page,
13        limit: result.limit,
14        totalPages: result.totalPages,
15      },
16    });
17   } catch (err) {
18     res.status(500).json({ success: false, error: { message: err.message } });
19   }
20 }
21
22 export async function handleGet(req, res) {
23   try {
24     const { code } = req.params;
25     const result = await getSalesPerson(code);
26     if (!result) return res.status(404).json({ success: false, error: { message: `Sales person not found:
    ${code}` } });
27     res.json({ success: true, data: result });
28   } catch (err) {
29     res.status(500).json({ success: false, error: { message: err.message } });
30   }
31 }
32
33 export async function handleCreate(req, res) {
34   try {
35     const { code, name, start_work_date } = req.body;
36     const result = await createSalesPerson({ code, name, start_work_date });
37     res.status(201).json({ success: true, data: result });
38   } catch (err) {
39     res.status(500).json({ success: false, error: { message: err.message } });
40   }
41 }
42
43 export async function handleUpdate(req, res) {
44   try {
45     const { code } = req.params;
46     const { name, start_work_date } = req.body;
47     const result = await updateSalesPerson(code, { name, start_work_date });
48     if (!result) return res.status(404).json({ success: false, error: { message: `Sales person not found:
    ${code}` } });
49     res.json({ success: true, data: result });
50   } catch (err) {
51     res.status(500).json({ success: false, error: { message: err.message } });
52   }
53 }
54
55 export async function handleDelete(req, res) {
56   try {
57     const { code } = req.params;
58     await deleteSalesPerson(code);
59     res.json({ success: true, data: { ok: true } });
60   } catch (err) {
61     res.status(500).json({ success: false, error: { message: err.message } });
62   }
63 }
64
```

Sales Person Routes

```
1 import { Router } from "express";
2 import { handleList, handleGet, handleCreate, handleUpdate, handleDelete } from "../controllers/
salesPersons.controller.js";
3
4 const router = Router();
5 router.get("/", handleList);
6 router.get("/:code", handleGet);
7 router.post("/", handleCreate);
8 router.put("/:code", handleUpdate);
9 router.delete("/:code", handleDelete);
10
11 export default router;
12
```

Configuration Service

```
1 import { pool } from "../db/pool.js";
2
3 export async function getConfig(key) {
4   const r = await pool.query("SELECT value FROM configuration WHERE key = $1", [key]);
5   if (r.rowCount === 0) throw new Error(`Config not found: ${key}`);
6   return r.rows[0];
7 }
8
```

Configuration Controller

```
1 import { getConfig } from "../services/configuration.service.js";
2
3 export async function handleGetConfig(req, res) {
4   try {
5     const { key } = req.params;
6     const result = await getConfig(key);
7     res.json({ success: true, data: result });
8   } catch (err) {
9     res.status(404).json({ success: false, error: { message: err.message } });
10  }
11 }
12
```

Configuration Routes

```
1 import { Router } from "express";
2 import { handleGetConfig } from "../controllers/configuration.controller.js";
3
4 const router = Router();
5 router.get("/:key", handleGetConfig);
6
7 export default router;
8
```


App Entry (mount /api/config)

```
1 // Backend entry: receives HTTP requests and forwards to routes (customers, products, invoices, reports)
2 import express from "express";
3 import cors from "cors";
4 import { execSync } from "child_process";
5 import path from "path";
6 import fs from "fs";
7
8 import logger from "../utils/logger.js";
9 import invoicesRoutes from "../routes/invoices.routes.js";
10 import salesPersonsRoutes from "../routes/salesPersons.routes.js";
11 import reportsRoutes from "../routes/reports.routes.js";
12 import customersRoutes from "../routes/customers.routes.js";
13 import productsRoutes from "../routes/products.routes.js";
14 import configurationRoutes from "../routes/configuration.routes.js";
15
16 const app = express();
17
18 // ANSI colors for request log (dev only; production uses JSON without codes)
19 const c = {
20   reset: "\x1b[0m",
21   dim: "\x1b[2m",
22   green: "\x1b[32m",
23   yellow: "\x1b[33m",
24   blue: "\x1b[34m",
25   magenta: "\x1b[35m",
26   cyan: "\x1b[36m",
27   red: "\x1b[31m",
28 };
29 const methodColor = { GET: c.green, POST: c.cyan, PUT: c.yellow, DELETE: c.magenta };
30 const statusCode = (s) => (s >= 500 ? c.red : s >= 400 ? c.yellow : s >= 300 ? c.blue : c.green);
31 const useColor = process.env.NODE_ENV !== "production" && process.stdout.isTTY;
32
33 // Request logging: one line per request, colored by method and status in dev
34 app.use((req, res, next) => {
35   const start = Date.now();
36   const method = req.method;
37   res.on("finish", () => {
38     const duration = Date.now() - start;
39     const status = res.statusCode;
40     let msg = `[${method}] ${req.originalUrl} ${status} ${duration}ms`;
41     if (useColor) {
42       const m = methodColor[method] || c.dim;
43       const s = statusCode(status);
44       msg = `${m}[${method}]${c.reset} ${req.originalUrl} ${s}${status}${c.reset} ${c.dim}${duration}ms${c.reset}`;
45     }
46     logger.info(msg, { method, url: req.originalUrl, status, durationMs: duration });
47   });
48   next();
49 });
50
51 // CORS: allow origin from CORS_ORIGIN env, or * if unset (e.g. dev). Allows common methods/headers.
52 const corsOrigin = process.env.CORS_ORIGIN;
53 app.use(
54   cors({
55     origin: corsOrigin === undefined || corsOrigin === "" ? true : corsOrigin.split(",").map((o) =>
56       o.trim()),
57     methods: ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
58     allowedHeaders: ["Content-Type", "Authorization"],
59   })
60 );
61 app.use(express.json());
62
63 // Health check
64 app.get("/health", (_, res) => res.json({ ok: true }));
65
66 // Check if repo has newer commits (for frontend update banner). Uses GIT_SHA from env, or git rev-parse
67 // when running dev.
68 const REPO = "llbumpbump/InvoiceDoc2";
69 const REPO_URL = `https://github.com/${REPO}`;
70 function getCurrentSha() {
71   const fromEnv = process.env.GIT_SHA || process.env.VERCEL_GIT_COMMIT_SHA ||
72     process.env.RENDER_GIT_COMMIT_SHA;
73   if (fromEnv) return fromEnv.trim();
74   let repoRoot = process.cwd();
75   if (!fs.existsSync(path.join(repoRoot, ".git"))) {
76     repoRoot = path.resolve(repoRoot, "..");
77   }
78 }
```

```

74   }
75   try {
76     return execSync("git rev-parse HEAD", { cwd: repoRoot, encoding: "utf8" }).trim();
77   } catch {
78     return null;
79   }
80 }
81 app.get("/api/updates-check", async (_, res) => {
82   const currentSha = getCurrentSha();
83   if (!currentSha) {
84     res.setHeader("X-Update-Available", "false");
85     return res.json({ updateAvailable: false });
86   }
87   try {
88     const r = await fetch(`https://api.github.com/repos/${REPO}/commits/main`, {
89       headers: { Accept: "application/vnd.github.v3+json" },
90     });
91     if (!r.ok) {
92       res.setHeader("X-Update-Available", "false");
93       return res.json({ updateAvailable: false });
94     }
95     const data = await r.json();
96     const latestSha = data?.sha;
97     const updateAvailable = !!latestSha && latestSha !== currentSha;
98     res.setHeader("X-Update-Available", updateAvailable ? "true" : "false");
99     if (updateAvailable) res.setHeader("X-Update-Repo-Url", REPO_URL);
100    res.json({ updateAvailable, repoUrl: REPO_URL });
101  } catch {
102    res.setHeader("X-Update-Available", "false");
103    res.json({ updateAvailable: false });
104  }
105 });
106
107 // API routes
108 app.use("/api/customers", customersRoutes);
109 app.use("/api/products", productsRoutes);
110 app.use("/api/invoices", invoicesRoutes);
111 app.use("/api/sales-persons", salesPersonsRoutes);
112 app.use("/api/reports", reportsRoutes);
113 app.use("/api/config", configurationRoutes);
114
115 const port = process.env.PORT || 4000;
116 const host = process.env.HOST || "0.0.0.0";
117 app.listen(port, host, () => logger.info(`Invoice server listening on ${host}:${port}`));
118

```

Invoice Service (discount calcs)

```

1 // Invoice CRUD: list (search, pagination, sort), get with line items, create/update/delete. Transactions
for create/update.
2 import { pool } from "../db/pool.js";
3
4 export async function listInvoices({
5   search = "",
6   page = 1,
7   limit = 10,
8   sortBy = "invoice_date",
9   sortDir = "desc",
10 } = {}) {
11   const offset = (Number(page) - 1) * Number(limit);
12
13   const allowedSort = ["invoice_no", "customer_name", "invoice_date", "amount_due"];
14   const sortColumn = allowedSort.includes(sortBy) ? sortBy : "invoice_date";
15   const sortDirection = sortDir === "asc" ? "ASC" : "DESC";
16
17   const searchParam = `%${search}%`;
18
19   const countResult = await pool.query(
20     `
21       SELECT COUNT(*) as total
22       FROM invoice i
23       JOIN customer c ON c.id = i.customer_id
24       WHERE i.invoice_no ILIKE $1 OR c.name ILIKE $1
25     `,
26     [searchParam],
27   );
28   const total = Number(countResult.rows[0].total);
29
30   const { rows } = await pool.query(
31     `
32       SELECT i.invoice_no, i.invoice_date, i.amount_due,
33             c.name as customer_name,
34             sp.name as sales_person_name
35       FROM invoice i
36       JOIN customer c ON c.id = i.customer_id
37       LEFT JOIN sales_person sp ON sp.id = i.sales_person_id
38       WHERE i.invoice_no ILIKE $1 OR c.name ILIKE $1
39       ORDER BY ${sortColumn} ${sortDirection} NULLS LAST, i.id DESC
40       LIMIT $2 OFFSET $3
41     `,
42     [searchParam, Number(limit), offset],
43   );
44
45   return {
46     data: rows,
47     total,
48     page: Number(page),
49     limit: Number(limit),
50     totalPages: Math.ceil(total / Number(limit)),
51   };
52 }
53
54 /** Resolve invoice_no to id (for internal use). */
55 async function resolveInvoiceId(invoice_no) {
56   const r = await pool.query("SELECT id FROM invoice WHERE invoice_no = $1", [invoice_no]);
57   return r.rowCount > 0 ? r.rows[0].id : null;
58 }
59
60 export async function getInvoice(idOrInvoiceNo) {
61   // Support both id (number) and invoice_no (string) for backward compatibility during migration
62   let id = idOrInvoiceNo;
63   if (typeof idOrInvoiceNo === "string" && String(idOrInvoiceNo).trim() !== "" &&
isNaN(Number(idOrInvoiceNo))) {
64     id = await resolveInvoiceId(String(idOrInvoiceNo).trim());
65     if (id == null) return null;
66   } else {
67     id = Number(idOrInvoiceNo);
68   }
69
70   const header = await pool.query(
71     `
72       select i.invoice_no, i.invoice_date, i.customer_id, i.total_amount, i.vat, i.amount_due,
73             i.vat_rate, i.total_price, i.total_discount, i.vat_amount,
74             c.code as customer_code, c.name as customer_name,
75             c.address_line1, c.address_line2,

```

```

76     co.name as country_name,
77     sp.code as sales_person_code,
78     sp.name as sales_person_name
79     from invoice i
80     join customer c on c.id = i.customer_id
81     left join country co on co.id = c.country_id
82     left join sales_person sp on sp.id = i.sales_person_id
83     where i.id = $1
84     `
85     [id],
86 );
87
88 if (header.rowCount === 0) return null;
89
90 const lines = await pool.query(
91     `
92     select li.id,
93           p.code as product_code, p.name as product_name,
94           u.code as units_code,
95           li.quantity, li.unit_price, li.extended_price,
96           li.line_discount_percent, li.line_discount_amount, li.line_net_price
97     from invoice_line_item li
98     join product p on p.id = li.product_id
99     join units u on u.id = p.units_id
100    where li.invoice_id = $1
101    order by li.id
102    `
103    [id],
104 );
105
106 return { header: header.rows[0], line_items: lines.rows };
107 }
108
109 /** Resolve product_code to id and compute discount fields. */
110 async function enrichLineItems(client, line_items) {
111     const enriched = [];
112     for (const li of line_items) {
113         const product_code = li.product_code != null ? String(li.product_code).trim() : null;
114         if (!product_code) throw new Error("Line item missing product_code");
115         const pr = await client.query(
116             "SELECT id, unit_price FROM product WHERE code = $1",
117             [product_code],
118         );
119         if (pr.rowCount === 0) throw new Error(`Product not found: ${product_code}`);
120         const product_id = pr.rows[0].id;
121         const unit_price = li.unit_price ?? Number(pr.rows[0].unit_price ?? 0);
122         const extended_price = Number(li.quantity) * Number(unit_price);
123         const line_discount_percent = Number(li.line_discount_percent || 0);
124         const line_discount_amount = Math.round(extended_price * line_discount_percent / 100 * 100) / 100;
125         const line_net_price = Math.round((extended_price - line_discount_amount) * 100) / 100;
126         enriched.push({ ...li, product_id, unit_price, extended_price, line_discount_percent,
127             line_discount_amount, line_net_price });
128     }
129     return enriched;
130 }
131 export async function createInvoice({ invoice_no, customer_code, sales_person_code, invoice_date,
132     vat_rate, line_items }) {
133     const client = await pool.connect();
134     try {
135         await client.query("begin");
136
137         const code = customer_code != null ? String(customer_code).trim() : "";
138         const cust = await client.query("SELECT id, credit_limit FROM customer WHERE code = $1", [code]);
139         if (cust.rowCount === 0) throw new Error(`Customer not found: ${code}`);
140         const customer_id = cust.rows[0].id;
141
142         // Resolve sales_person_code to id
143         let sales_person_id = null;
144         if (sales_person_code) {
145             const sp = await client.query("SELECT id FROM sales_person WHERE code = $1", [sales_person_code]);
146             if (sp.rowCount === 0) throw new Error(`Sales person not found: ${sales_person_code}`);
147             sales_person_id = sp.rows[0].id;
148         }
149
150         let resolvedInvoiceNo = invoice_no;
151         if (!resolvedInvoiceNo || String(resolvedInvoiceNo).trim() === "") {
152             const maxRes = await client.query("SELECT MAX(id) as m FROM invoice");
153             const nextId = (maxRes.rows[0].m || 0) + 1;

```

```

153     resolvedInvoiceNo = `INV-${nextId.toString().padStart(3, "0")}`;
154 }
155
156 const enriched = await enrichLineItems(client, line_items);
157
158 const total_price = enriched.reduce((s, x) => s + x.extended_price, 0);
159 const total_discount = enriched.reduce((s, x) => s + x.line_discount_amount, 0);
160 const vat_amount = Math.round((total_price - total_discount) * Number(vat_rate) * 100) / 100;
161 const amount_due = (total_price - total_discount) + vat_amount;
162
163 if (cust.rows[0].credit_limit != null) {
164     const limit = Number(cust.rows[0].credit_limit);
165     if (amount_due > limit) {
166         throw new Error(`Amount due (${amount_due}) exceeds customer credit limit (${limit}).`);
167     }
168 }
169
170 const inv = await client.query(
171     `
172         insert into invoice (id, created_at, invoice_no, invoice_date, customer_id, sales_person_id,
173             total_amount, vat, amount_due, vat_rate, total_price, total_discount, vat_amount)
174         values (
175             (select coalesce(max(id),0)+1 from invoice),
176             now(),
177             $1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11
178         )
179         returning id, invoice_no
180     `,
181     [resolvedInvoiceNo, invoice_date, customer_id, sales_person_id,
182         total_price, vat_amount, amount_due, Number(vat_rate), total_price, total_discount, vat_amount],
183 );
184
185 const invoice_id = inv.rows[0].id;
186
187 for (const li of enriched) {
188     await client.query(
189         `
190             insert into invoice_line_item (id, created_at, invoice_id, product_id, quantity, unit_price,
191             extended_price,
192             line_discount_percent, line_discount_amount, line_net_price)
193         values (
194             (select coalesce(max(id),0)+1 from invoice_line_item),
195             now(),
196             $1,$2,$3,$4,$5,$6,$7,$8
197         )
198         `,
199         [invoice_id, li.product_id, li.quantity, li.unit_price, li.extended_price,
200             li.line_discount_percent, li.line_discount_amount, li.line_net_price],
201     );
202 }
203
204 await client.query("commit");
205 return { invoice_no: inv.rows[0].invoice_no };
206 } catch (err) {
207     await client.query("rollback");
208     throw err;
209 } finally {
210     client.release();
211 }
212
213 export async function deleteInvoice(idOrInvoiceNo) {
214     let id = idOrInvoiceNo;
215     if (typeof idOrInvoiceNo === "string" && String(idOrInvoiceNo).trim() !== "" &&
216         isNaN(Number(idOrInvoiceNo))) {
217         id = await resolveInvoiceId(String(idOrInvoiceNo).trim());
218         if (id == null) return null;
219     } else {
220         id = Number(idOrInvoiceNo);
221     }
222     await pool.query("delete from invoice where id=$1", [id]);
223     return { ok: true };
224 }
225
226 export async function updateInvoice(
227     idOrInvoiceNo,
228     { invoice_no, customer_code, sales_person_code, invoice_date, vat_rate, line_items },
229 ) {
230     let id = idOrInvoiceNo;

```

```

230   if (typeof idOrInvoiceNo === "string" && String(idOrInvoiceNo).trim() !== "" &&
isNaN(Number(idOrInvoiceNo))) {
231       id = await resolveInvoiceId(String(idOrInvoiceNo).trim());
232       if (id == null) return null;
233   } else {
234       id = Number(idOrInvoiceNo);
235   }
236
237   const code = customer_code !== null ? String(customer_code).trim() : "";
238   const cust = await pool.query("SELECT id, credit_limit FROM customer WHERE code = $1", [code]);
239   if (cust.rowCount === 0) throw new Error(`Customer not found: ${code}`);
240   const customer_id = cust.rows[0].id;
241
242   const client = await pool.connect();
243   try {
244       await client.query("begin");
245
246       // Resolve sales_person_code !' id
247       let sales_person_id = null;
248       if (sales_person_code) {
249           const sp = await client.query("SELECT id FROM sales_person WHERE code = $1", [sales_person_code]);
250           if (sp.rowCount === 0) throw new Error(`Sales person not found: ${sales_person_code}`);
251           sales_person_id = sp.rows[0].id;
252       }
253
254       const enriched = await enrichLineItems(client, line_items);
255
256       const total_price = enriched.reduce((s, x) => s + x.extended_price, 0);
257       const total_discount = enriched.reduce((s, x) => s + x.line_discount_amount, 0);
258       const vat_amount = Math.round((total_price - total_discount) * Number(vat_rate) * 100) / 100;
259       const amount_due = (total_price - total_discount) + vat_amount;
260
261       if (cust.rows[0].credit_limit !== null) {
262           const limit = Number(cust.rows[0].credit_limit);
263           if (amount_due > limit) {
264               throw new Error(`Amount due (${amount_due}) exceeds customer credit limit (${limit}).`);
265           }
266       }
267
268       let resolvedInvoiceNo = (invoice_no !== null && String(invoice_no).trim() !== "") ?
String(invoice_no).trim() : null;
269       if (resolvedInvoiceNo === null) {
270           const cur = await client.query("SELECT invoice_no FROM invoice WHERE id=$1", [id]);
271           if (cur.rowCount > 0) resolvedInvoiceNo = cur.rows[0].invoice_no;
272           else resolvedInvoiceNo = `INV-${id}`;
273       }
274
275       await client.query(
276           `
277           UPDATE invoice
278           SET invoice_no=$1, invoice_date=$2, customer_id=$3, sales_person_id=$4,
279             total_amount=$5, vat=$6, amount_due=$7,
280             vat_rate=$8, total_price=$9, total_discount=$10, vat_amount=$11
281           WHERE id=$12
282           `,
283           [resolvedInvoiceNo, invoice_date, customer_id, sales_person_id,
284             total_price, vat_amount, amount_due, Number(vat_rate), total_price, total_discount, vat_amount, id],
285       );
286
287       const keptLineIds = line_items.filter((li) => li.id !== null && Number(li.id) > 0).map((li) =>
Number(li.id));
288
289       if (keptLineIds.length > 0) {
290           await client.query(
291               "DELETE FROM invoice_line_item WHERE invoice_id = $1 AND id != ALL($2::bigint[])",
292               [id, keptLineIds],
293           );
294       } else {
295           await client.query("DELETE FROM invoice_line_item WHERE invoice_id = $1", [id]);
296       }
297
298       for (const li of enriched) {
299           const lineId = li.id !== null && Number(li.id) > 0 ? Number(li.id) : null;
300           if (lineId) {
301               await client.query(
302                   `
303                   UPDATE invoice_line_item
304                   SET product_id=$1, quantity=$2, unit_price=$3, extended_price=$4,
305                     line_discount_percent=$5, line_discount_amount=$6, line_net_price=$7

```

```

306         WHERE id=$8 AND invoice_id=$9
307     `
308     [li.product_id, li.quantity, li.unit_price, li.extended_price,
309     li.line_discount_percent, li.line_discount_amount, li.line_net_price, lineId, id],
310 );
311 } else {
312     await client.query(
313     `
314         INSERT INTO invoice_line_item (id, created_at, invoice_id, product_id, quantity, unit_price,
extended_price,
315         line_discount_percent, line_discount_amount, line_net_price)
316         VALUES (
317         (select coalesce(max(id),0)+1 from invoice_line_item),
318         now(),
319         $1,$2,$3,$4,$5,$6,$7,$8
320         )
321     `
322     [id, li.product_id, li.quantity, li.unit_price, li.extended_price,
323     li.line_discount_percent, li.line_discount_amount, li.line_net_price],
324 );
325 }
326 }
327
328 await client.query("commit");
329 const inv = await pool.query("SELECT invoice_no FROM invoice WHERE id = $1", [id]);
330 return { invoice_no: inv.rows[0]?.invoice_no };
331 } catch (err) {
332     await client.query("rollback");
333     throw err;
334 } finally {
335     client.release();
336 }
337 }
338

```

Client Configuration API

```
1 import { http } from './http.js';
2
3 export async function getConfig(key) {
4   const res = await http(`/api/config/${encodeURIComponent(key)}`);
5   return res.data.value;
6 }
7
```


Client Sales Person API (CRUD)

```
1 import { http } from "../http.js";
2
3 export async function listSalesPersons(params = {}) {
4   const query = new URLSearchParams(params).toString();
5   const res = await http(`/api/sales-persons${query} ? `?${query}` : ""`);
6   return { data: res.data, ...(res.meta || {}) };
7 }
8
9 export async function getSalesPerson(code) {
10   const res = await http(`/api/sales-persons/${encodeURIComponent(code)}`);
11   return res.data;
12 }
13
14 export async function createSalesPerson(body) {
15   const res = await http("/api/sales-persons", { method: "POST", body: JSON.stringify(body) });
16   return res.data;
17 }
18
19 export async function updateSalesPerson(code, body) {
20   const res = await http(`/api/sales-persons/${encodeURIComponent(code)}`, { method: "PUT", body:
JSON.stringify(body) });
21   return res.data;
22 }
23
24 export async function deleteSalesPerson(code) {
25   const res = await http(`/api/sales-persons/${encodeURIComponent(code)}`, { method: "DELETE" });
26   return res.data;
27 }
28
```

Sales Person List Page

```
1 import React from "react";
2 import { toast } from "react-toastify";
3 import { listSalesPersons, deleteSalesPerson } from "../../api/salesPersons.api.js";
4 import DataList from "../../components/DataList.jsx";
5 import { ConfirmModal } from "../../components/Modal.jsx";
6
7 export default function SalesPersonList() {
8   const fetchData = React.useCallback((params) => listSalesPersons(params), []);
9   const [confirmModal, setConfirmModal] = React.useState({ isOpen: false, id: null });
10  const [refreshTrigger, setRefreshTrigger] = React.useState(0);
11
12  const handleDelete = (id) => {
13    setConfirmModal({ isOpen: true, id });
14  };
15
16  const confirmDelete = async () => {
17    try {
18      await deleteSalesPerson(confirmModal.id);
19      setConfirmModal({ isOpen: false, id: null });
20      setRefreshTrigger((t) => t + 1);
21      toast.success("Sales person deleted.");
22    } catch (e) {
23      toast.error(String(e.message || e));
24      setConfirmModal({ isOpen: false, id: null });
25    }
26  };
27
28  const columns = [
29    { key: "code", label: "Code" },
30    { key: "name", label: "Name" },
31    { key: "start_work_date", label: "Start Date", render: (v) => v ? String(v).slice(0, 10) : "-" },
32  ];
33
34  return (
35    <>
36      <ConfirmModal
37        isOpen={confirmModal.isOpen}
38        onClose={() => setConfirmModal({ isOpen: false, id: null })}
39        onConfirm={confirmDelete}
40        closeOnConfirm={false}
41        title="Delete Sales Person"
42        message="Are you sure you want to delete this sales person?"
43        confirmText="Delete"
44      />
45      <DataList
46        title="Sales Persons"
47        fetchData={fetchData}
48        columns={columns}
49        searchPlaceholder="Search code, name..."
50        itemName="sales persons"
51        basePath="/sales-persons"
52        itemKey="code"
53        onDelete={handleDelete}
54        refreshTrigger={refreshTrigger}
55      />
56    </>
57  );
58 }
59
```

Sales Person Create/Edit/View Page

```
1 import React from "react";
2 import { useNavigate, Link, useParams } from "react-router-dom";
3 import { toast } from "react-toastify";
4 import { getSalesPerson, createSalesPerson, updateSalesPerson } from "../../api/salesPersons.api.js";
5 import Loading from "../../components/Loading.jsx";
6
7 export default function SalesPersonPage({ mode: propMode }) {
8   const { id } = useParams();
9   const mode = propMode || (id ? "view" : "create");
10  const nav = useNavigate();
11
12  const [code, setCode] = React.useState("");
13  const [name, setName] = React.useState("");
14  const [startWorkDate, setStartWorkDate] = React.useState("");
15  const [autoCode, setAutoCode] = React.useState(true);
16  const [err, setErr] = React.useState("");
17  const [submitting, setSubmitting] = React.useState(false);
18  const [loading, setLoading] = React.useState(mode !== "create");
19
20  React.useEffect(() => {
21    if (mode !== "create") {
22      getSalesPerson(id)
23        .then((sp) => {
24          if (!sp) { setErr("Sales person not found"); setLoading(false); return; }
25          setCode(sp.code || "");
26          setName(sp.name || "");
27          setStartWorkDate(sp.start_work_date ? String(sp.start_work_date).slice(0, 10) : "");
28          setLoading(false);
29        })
30        .catch((e) => {
31          setErr(String(e.message || e));
32          setLoading(false);
33        });
34    }
35  }, [id, mode]);
36
37  async function handleSubmit(e) {
38    e.preventDefault();
39    setErr("");
40    setSubmitting(true);
41    try {
42      const payload = { name, start_work_date: startWorkDate || null };
43      if (mode === "create") {
44        if (!autoCode) payload.code = code;
45        await createSalesPerson(payload);
46        toast.success("Sales person created.");
47      } else {
48        await updateSalesPerson(id, payload);
49        toast.success("Sales person updated.");
50      }
51      nav("/sales-persons");
52    } catch (e) {
53      const msg = String(e.message || e);
54      setErr(msg);
55      toast.error(msg);
56    } finally {
57      setSubmitting(false);
58    }
59  }
60
61  if (loading) return <Loading size="large" />;
62
63  const isView = mode === "view";
64  const isCreate = mode === "create";
65  const titles = { create: "Create Sales Person", view: "Sales Person Details", edit: "Edit Sales
Person" };
66  const title = titles[mode];
67
68  if (isView) {
69    return (
70      <div>
71        <div className="page-header">
72          <h3 className="page-title">{title}</h3>
73          <div className="flex gap-4">
74            <Link to="/sales-persons" className="btn btn-outline">!• Back</Link>
75            <Link to={` /sales-persons/${id}/edit`} className="btn btn-primary">Edit</Link>
76          </div>
77        </div>
78      </div>
79    );
80  }
```

```

77         </div>
78         <div className="card">
79             <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: "2rem" }}>
80                 <div>
81                     <h4 style={{ marginBottom: "1.5rem", color: "var(--primary)" }}>Basic
Information</h4>
82                     <div style={{ marginBottom: "1rem" }}>
83                         <div style={{ fontSize: "0.8rem", color: "var(--text-muted)",
marginBottom: "4px" }}>Code</div>
84                             <div style={{ fontWeight: 600, fontSize: "1.1rem" }}>{code}</div>
85                         </div>
86                         <div style={{ marginBottom: "1rem" }}>
87                             <div style={{ fontSize: "0.8rem", color: "var(--text-muted)",
marginBottom: "4px" }}>Name</div>
88                             <div style={{ fontWeight: 600, fontSize: "1.1rem" }}>{name}</div>
89                         </div>
90                         <div style={{ marginBottom: "1rem" }}>
91                             <div style={{ fontSize: "0.8rem", color: "var(--text-muted)",
marginBottom: "4px" }}>Start Date</div>
92                             <div>{startWorkDate || "-"}</div>
93                         </div>
94                     </div>
95                 </div>
96             </div>
97         </div>
98     );
99 }
100
101     return (
102         <div>
103             <div className="page-header">
104                 <h3 className="page-title">{title}</h3>
105                 <Link to="/sales-persons" className="btn btn-outline">
106                     <svg style={{ marginRight: 8 }} width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"><line x1="19" y1="12" x2="5"
y2="12"></line><polyline points="12 19 5 12 12 5"></polyline></svg>
107                     Back
108                 </Link>
109             </div>
110             {err && <div className="alert alert-error">{err}</div>}
111             <div className="card">
112                 <form onSubmit={handleSubmit}>
113                     <div style={{ display: "grid", gridTemplateColumns: "1fr 2fr", gap: "1rem",
marginBottom: "1rem" }}>
114                         <div className="form-group">
115                             <label className="form-label">{isCreate && autoCode ? "Code" : <>Code <span
className="required-marker">*</span></></label>
116                             {isCreate ? (
117                                 <div className="flex gap-2">
118                                     <input
119                                         className="form-control"
120                                         disabled={autoCode}
121                                         value={code}
122                                         onChange={(e) => setCode(e.target.value)}
123                                         placeholder="SP001"
124                                     />
125                                     <div className="form-inline-option">
126                                         <input type="checkbox" checked={autoCode} onChange={(e) =>
setAutoCode(e.target.checked)} id="sp_auto" />
127                                         <label htmlFor="sp_auto">Auto</label>
128                                     </div>
129                                 </div>
130                             ) : (
131                                 <input className="form-control" value={code} disabled readOnly
placeholder="SP001" />
132                             )}
133                         </div>
134                         <div className="form-group">
135                             <label className="form-label">Name <span className="required-marker">*</span></
label>
136                             <input className="form-control" required value={name} onChange={(e) =>
setName(e.target.value)} placeholder="Sales Person Name" />
137                         </div>
138                     </div>
139                     <div style={{ display: "grid", gridTemplateColumns: "1fr 2fr", gap: "1rem",
marginBottom: "1.5rem" }}>
140                         <div className="form-group">
141                             <label className="form-label">Start Date</label>
142                             <input type="date" className="form-control" value={startWorkDate}
onChange={(e) => setStartWorkDate(e.target.value)} />

```

```
143             </div>
144         </div>
145         <button type="submit" className="btn btn-primary" disabled={submitting}>
146             {submitting ? (isCreate ? "Creating..." : "Saving...") : (isCreate ? "Create Sales
Person" : "Save Changes")}
147         </button>
148     </form>
149 </div>
150 </div>
151     );
152 }
153
```

App Router & Nav (Sales Persons)

```
1 // App entry: defines which URL path renders which page (routes)
2 import React from "react";
3 import ReactDOM from "react-dom/client";
4 import { BrowserRouter, Routes, Route, NavLink, Navigate } from "react-router-dom";
5 import { ToastContainer } from "react-toastify";
6 import "react-toastify/dist/ReactToastify.css";
7 import InvoiceList from "../pages/invoices/InvoiceList.jsx";
8 import InvoicePage from "../pages/invoices/InvoicePage.jsx";
9 import CustomerList from "../pages/customers/CustomerList.jsx";
10 import CustomerPage from "../pages/customers/CustomerPage.jsx";
11 import ProductList from "../pages/products/ProductList.jsx";
12 import ProductPage from "../pages/products/ProductPage.jsx";
13 import Reports from "../pages/reports/Reports.jsx";
14 import SalesPersonList from "../pages/salesPersons/SalesPersonList.jsx";
15 import SalesPersonPage from "../pages/salesPersons/SalesPersonPage.jsx";
16 import { http } from "../api/http.js";
17 import "../index.css";
18
19 // Collapsible submenu (e.g. Reports !' Product Sales, Monthly Sales)
20 function SubMenu({ icon, label, children, basePath }) {
21   const location = window.location.pathname;
22   const isActive = location.startsWith(basePath);
23   const [isOpen, setIsOpen] = React.useState(isActive);
24
25   return (
26     <div className={`nav-group ${isOpen ? 'open' : ''}`}>
27       <button
28         type="button"
29         className={`nav-item nav-group-toggle ${isActive ? 'active' : ''}`}
30         onClick={() => setIsOpen(!isOpen)}
31       >
32         {icon}
33         <span style={{ flex: 1 }}>{label}</span>
34         <svg
35           width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor"
36           strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"
37           style={{ transition: 'transform 0.2s', transform: isOpen ? 'rotate(180deg)' : 'rotate(0)' }}
38         >
39           <polyline points="6 9 12 15 18 9"></polyline>
40         </svg>
41       </button>
42       <div className="nav-submenu" style={{ display: isOpen ? 'block' : 'none' }}>
43         {children}
44       </div>
45     </div>
46   );
47 }
48
49 function Sidebar() {
50   // NavLink injects isActive when the current URL matches this link
51   const getLinkClass = ({ isActive }) => isActive ? "nav-item active" : "nav-item";
52   const getSubLinkClass = ({ isActive }) => isActive ? "nav-subitem active" : "nav-subitem";
53
54   return (
55     <aside className="sidebar no-print">
56       <div className="sidebar-header">
57         <div className="brand">
58           <svg width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="currentColor"
59           strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
60             <path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2 2v8z"></path>
61             <polyline points="14 2 14 8 20 8"></polyline>
62             <line x1="16" y1="13" x2="8" y2="13"></line>
63             <line x1="16" y1="17" x2="8" y2="17"></line>
64             <polyline points="10 9 9 9 8 9"></polyline>
65           </svg>
66           InvoiceDoc v2
67         </div>
68       </div>
69       <nav className="sidebar-nav">
70         <NavLink to="/invoices" className={getLinkClass}>
71           <svg style={{ marginRight: 10 }} width="18" height="18" viewBox="0 0 24 24" fill="none"
72           stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"><rect x="3" y="4" width="18"
73           height="18" rx="2" ry="2"></rect><line x1="16" y1="2" x2="16" y2="6"></line><line x1="8" y1="2" x2="8" y2="6"></line><line x1="3" y1="10" x2="21" y2="10"></line></svg>
74           Invoices
75         </NavLink>
76         <NavLink to="/customers" className={getLinkClass}>
```

```

74         <svg style={{ marginRight: 10 }} width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"><path d="M17 21v-2a4 4 0 0
0-4-4H5a4 4 0 0 0-4 4v2"></path><circle cx="9" cy="7" r="4"></circle><path d="M23 21v-2a4 4 0 0 0-3-3.87"></
path><path d="M16 3.13a4 4 0 0 1 0 7.75"></path></svg>
75         Customers
76     </NavLink>
77     <NavLink to="/products" className={getLinkClass}>
78         <svg style={{ marginRight: 10 }} width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"><path d="M21 16v8a2 2 0 0
0-1-1.73l-7-4a2 2 0 0 0-2 0l-7 4A2 2 0 0 0 3 8v8a2 2 0 0 0 1 1.73l7 4a2 2 0 0 0 2 0l7-4A2 2 0 0 0 21 16z"></
path><polyline points="3.27 6.96 12 12.01 20.73 6.96"></polyline><line x1="12" y1="22.08" x2="12" y2="12"></
line></svg>
79         Products
80     </NavLink>
81     <NavLink to="/sales-persons" className={getLinkClass}>
82         <svg style={{ marginRight: 10 }} width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"><circle cx="12" cy="8"
r="4"></circle><path d="M4 20c0-4 3.6-7 8-7s8 3 8 7"></path></svg>
83         Sales Persons
84     </NavLink>
85
86     <SubMenu
87         basePath="/reports"
88         icon={<svg style={{ marginRight: 10 }} width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"><line x1="18" y1="20"
x2="18" y2="10"></line><line x1="12" y1="20" x2="12" y2="4"></line><line x1="6" y1="20" x2="6" y2="14"></line></
svg>}
89         label="Reports"
90     >
91         <NavLink to="/reports/product-sales" className={getSubLinkClass}>
92             Product Sales
93         </NavLink>
94         <NavLink to="/reports/monthly-sales" className={getSubLinkClass}>
95             Monthly Sales
96         </NavLink>
97         <NavLink to="/reports/customer-sales" className={getSubLinkClass}>
98             Customer Buying
99         </NavLink>
100     </SubMenu>
101 </nav>
102 </aside>
103 );
104 }
105
106 // Banner when repo has newer commits (server returns updateAvailable if GIT_SHA is set and behind GitHub)
107 function UpdateBanner() {
108     const [show, setShow] = React.useState(false);
109     const [repoUrl, setRepoUrl] = React.useState("");
110     const DISMISS_KEY = "invoicedoc2-update-banner-dismissed";
111
112     React.useEffect(() => {
113         if (localStorage.getItem(DISMISS_KEY)) return;
114         http("/api/updates-check")
115             .then((data) => {
116                 if (data.updateAvailable && data.repoUrl) {
117                     setShow(true);
118                     setRepoUrl(data.repoUrl);
119                 }
120             })
121             .catch(() => {});
122     }, []);
123
124     const dismiss = () => {
125         localStorage.setItem(DISMISS_KEY, "1");
126         setShow(false);
127     };
128
129     if (!show) return null;
130     return (
131         <div className="update-banner no-print">
132             <span>
133                 Update available: the repo has new changes – run <code>git pull origin</code> in your project
134                 folder to get the latest, or check{" "}
135                 <a href={repoUrl} target="_blank" rel="noreferrer">
136                     GitHub
137                 </a>
138             </span>
139             <button type="button" className="update-banner-dismiss" onClick={dismiss} aria-label="Close">
140                 x
141             </button>
142         </div>
143     );
144 }

```

```

140     </button>
141   </div>
142 );
143 }
144
145 // Mobile warning overlay
146 function MobileWarning() {
147   const [dismissed, setDismissed] = React.useState(false);
148   if (dismissed) return null;
149
150   return (
151     <div className="mobile-warning">
152       <div className="mobile-warning-content">
153         <svg width="48" height="48" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2"
strokeLinecap="round" strokeLinejoin="round">
154           <rect x="5" y="2" width="14" height="20" rx="2" ry="2"></rect>
155           <line x1="12" y1="18" x2="12.01" y2="18"></line>
156         </svg>
157         <h3>Whoa there! ðŸ’€</h3>
158         <p>TA was too busy touching grass to make this mobile-friendly.<br/>Please grab a laptop. We
believe in you.</p>
159         <button className="btn btn-primary" onClick={() => setDismissed(true)}>
160           Okay, I'll suffer
161         </button>
162       </div>
163     </div>
164   );
165 }
166
167 function Layout({ children }) {
168   return (
169     <div className="layout-container">
170       <ToastContainer position="bottom-right" autoClose={4000} hideProgressBar={false} theme="light" />
171       <MobileWarning />
172       <UpdateBanner />
173       <div className="top-banner no-print">
174         This is a sample term project for CPE241 Database Systems. Some functions may be incomplete. Click
175         <a href="https://google.com" target="_blank" rel="noreferrer">here for questions</a>
176       </div>
177       <div className="app-layout">
178         <Sidebar />
179         <main className="main-wrapper">
180           {children}
181         </main>
182       </div>
183     </div>
184   );
185 }
186
187 // Route definitions: path !' component (e.g. /invoices !' InvoiceList)
188 ReactDOM.createRoot(document.getElementById("root")).render(
189   <React.StrictMode>
190     <BrowserRouter>
191       <Routes>
192         {/* Default route !' invoices list */}
193         <Route path="/" element={<Navigate to="/invoices" replace />} />
194         <Route path="/invoices" element={<Layout><InvoiceList /></Layout>} />
195         <Route path="/invoices/new" element={<Layout><InvoicePage mode="create" /></Layout>} />
196         <Route path="/invoices/:id" element={<Layout><InvoicePage mode="view" /></Layout>} />
197         <Route path="/invoices/:id/edit" element={<Layout><InvoicePage mode="edit" /></Layout>} />
198         <Route path="/customers" element={<Layout><CustomerList /></Layout>} />
199         <Route path="/customers/new" element={<Layout><CustomerPage mode="create" /></Layout>} />
200         <Route path="/customers/:id" element={<Layout><CustomerPage mode="view" /></Layout>} />
201         <Route path="/customers/:id/edit" element={<Layout><CustomerPage mode="edit" /></Layout>} />
202         <Route path="/products" element={<Layout><ProductList /></Layout>} />
203         <Route path="/products/new" element={<Layout><ProductPage mode="create" /></Layout>} />
204         <Route path="/products/:id" element={<Layout><ProductPage mode="view" /></Layout>} />
205         <Route path="/products/:id/edit" element={<Layout><ProductPage mode="edit" /></Layout>} />
206         <Route path="/sales-persons" element={<Layout><SalesPersonList /></Layout>} />
207         <Route path="/sales-persons/new" element={<Layout><SalesPersonPage mode="create" /></Layout>} />
208         <Route path="/sales-persons/:id" element={<Layout><SalesPersonPage mode="view" /></Layout>} />
209         <Route path="/sales-persons/:id/edit" element={<Layout><SalesPersonPage mode="edit" /></Layout>} />
210         <Route path="/reports" element={<Navigate to="/reports/product-sales" replace />} />
211         <Route path="/reports/product-sales" element={<Layout><Reports type="product-sales" /></Layout>} />
212         <Route path="/reports/monthly-sales" element={<Layout><Reports type="monthly-sales" /></Layout>} />
213         <Route path="/reports/customer-sales" element={<Layout><Reports type="customer-sales" /></
Layout>} />
214       </Routes>
215     </BrowserRouter>

```



```
216     </React.StrictMode>
217   );
218
```

Line Items Editor (discount columns)

```
1 // Line items table: add/remove/reorder rows, insert row between. Drag and drop and up/down buttons.
2 // Each row is a separate component (LineItemRow.jsx) for easier reading.
3 import React, { useState } from "react";
4 import { toast } from "react-toastify";
5 import { formatBaht } from "../utils.js";
6 import { getProduct } from "../api/products.api.js";
7 import ProductPickerModal from "../ProductPickerModal.jsx";
8 import LineItemRow from "../LineItemRow.jsx";
9
10 export default function LineItemsEditor({ value, onChange }) {
11   const items = value;
12   const [dragIndex, setDragIndex] = useState(null);
13   const [dragOverIndex, setDragOverIndex] = useState(null);
14   const [productPickerOpen, setProductPickerOpen] = useState(false);
15   const [pickerRowIndex, setPickerRowIndex] = useState(0);
16   const [productErrorRow, setProductErrorRow] = useState(null); // row index that has "product not found"
17
18   // Update one row by index
19   function update(i, patch) {
20     const next = items.map((x, idx) => (idx === i ? { ...x, ...patch } : x));
21     onChange(next);
22   }
23
24   // Add a new row with empty product (user must select) at the end
25   function addRow() {
26     onChange([
27       ...items,
28       { product_code: "", product_name: "", quantity: 1, unit_price: 0, line_discount_percent: 0 },
29     ]);
30   }
31
32   // Insert a new empty row after index i (add row in between)
33   function insertRowAfter(i) {
34     const newRow = { product_code: "", product_name: "", quantity: 1, unit_price: 0,
line_discount_percent: 0 };
35     const next = [...items.slice(0, i + 1), newRow, ...items.slice(i + 1)];
36     onChange(next);
37   }
38
39   // Remove a row by index
40   function removeRow(i) {
41     onChange(items.filter((_, idx) => idx !== i));
42   }
43
44   // Move item up
45   function moveUp(i) {
46     if (i <= 0) return;
47     const newItems = [...items];
48     [newItems[i - 1], newItems[i]] = [newItems[i], newItems[i - 1]];
49     onChange(newItems);
50   }
51
52   // Move item down
53   function moveDown(i) {
54     if (i >= items.length - 1) return;
55     const newItems = [...items];
56     [newItems[i], newItems[i + 1]] = [newItems[i + 1], newItems[i]];
57     onChange(newItems);
58   }
59
60   // Drag and drop handlers
61   function handleDragStart(e, index) {
62     setDragIndex(index);
63     e.dataTransfer.effectAllowed = 'move';
64     e.dataTransfer.setData('text/plain', index);
65   }
66
67   function handleDragOver(e, index) {
68     e.preventDefault();
69     e.dataTransfer.dropEffect = 'move';
70     setDragOverIndex(index);
71   }
72
73   function handleDragLeave() {
74     setDragOverIndex(null);
75   }
76 }
```

```

77     function handleDrop(e, dropIndex) {
78         e.preventDefault();
79         if (dragIndex === null || dragIndex === dropIndex) {
80             setDragIndex(null);
81             setDragOverIndex(null);
82             return;
83         }
84
85         const newItems = [...items];
86         const [removed] = newItems.splice(dragIndex, 1);
87         newItems.splice(dropIndex, 0, removed);
88         onChange(newItems);
89
90         setDragIndex(null);
91         setDragOverIndex(null);
92     }
93
94     function handleDragEnd() {
95         setDragIndex(null);
96         setDragOverIndex(null);
97     }
98
99     function onPickProduct(i, productCode, productData) {
100         setProductErrorRow(null);
101         if (!productData) {
102             update(i, { product_code: "", product_name: "", product_label: "", units_code: "", unit_price:
0 });
103             return;
104         }
105
106         const existingIndex = items.findIndex((it, idx) => idx !== i && String(it.product_code) ===
String(productData.code));
107         if (existingIndex !== -1) {
108             const currentQty = Number(items[i].quantity || 1);
109             const existingQty = Number(items[existingIndex].quantity || 0);
110             const newItems = items.filter((_, idx) => idx !== i);
111             newItems[existingIndex > i ? existingIndex - 1 : existingIndex] = {
112                 ...newItems[existingIndex > i ? existingIndex - 1 : existingIndex],
113                 quantity: existingQty + currentQty
114             };
115             onChange(newItems);
116         } else {
117             update(i, {
118                 product_code: productData.code,
119                 product_name: productData.name ?? "",
120                 product_label: productData.label,
121                 units_code: productData.units_code,
122                 unit_price: Number(productData.unit_price || 0)
123             });
124         }
125     }
126
127     function handleProductCodeBlur(i) {
128         const code = String(items[i]?.product_code ?? "").trim();
129         setProductErrorRow(null);
130         if (!code) return;
131         getProduct(code)
132             .then((p) => {
133                 update(i, {
134                     product_code: p.code,
135                     product_name: p.name ?? "",
136                     product_label: `${p.code} - ${p.name}`,
137                     units_code: p.units_code ?? "",
138                     unit_price: Number(p.unit_price ?? 0)
139                 });
140             })
141             .catch(() => {
142                 update(i, { product_name: "", product_label: "", units_code: "", unit_price: 0 });
143                 setProductErrorRow(i);
144                 toast.error(`Product not found: ${code}`);
145             });
146     }
147
148     function computeExtended(it) {
149         const q = Number(it.quantity || 0);
150         const up = Number(it.unit_price || 0);
151         return q * up;
152     }
153

```

```

154     function computeDiscountAmount(it) {
155         const extended = computeExtended(it);
156         const pct = Number(it.line_discount_percent || 0);
157         return Math.round(extended * pct / 100 * 100) / 100;
158     }
159
160     function computeNetPrice(it) {
161         return Math.round((computeExtended(it) - computeDiscountAmount(it)) * 100) / 100;
162     }
163
164     const totalPrice = items.reduce((s, it) => s + computeExtended(it), 0);
165     const totalDiscount = items.reduce((s, it) => s + computeDiscountAmount(it), 0);
166
167     return (
168         <div className="card">
169             <div style={{
170                 display: "flex",
171                 justifyContent: "space-between",
172                 alignItems: "center",
173                 marginBottom: 12,
174                 paddingBottom: 12,
175                 borderBottom: '1px solid var(--border)'
176             }}>
177                 <div>
178                     <h4 style={{ margin: 0, fontSize: '0.95rem', fontWeight: 600, color: 'var(--text-
main)' }}>Line Items</h4>
179                     <span style={{ fontSize: '0.8rem', color: 'var(--text-muted)' }}>
180                         {items.length} item{items.length !== 1 ? 's' : ''} added • Drag to reorder
181                     </span>
182                 </div>
183                 <button
184                     type="button"
185                     onClick={addRow}
186                     className="btn btn-primary"
187                     style={{ padding: '8px 16px' }}
188                 >
189                     <svg style={{ marginRight: 6 }} width="14" height="14" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2.5" strokeLinecap="round" strokeLinejoin="round">
190                         <line x1="12" y1="5" x2="12" y2="19"></line>
191                         <line x1="5" y1="12" x2="19" y2="12"></line>
192                     </svg>
193                     Add Item
194                 </button>
195             </div>
196
197             <div className="table-container">
198                 <table className="modern-table" style={{ fontSize: '0.9rem' }}>
199                     <thead>
200                         <tr>
201                             <th style={{ width: '60px' }} className="text-center">#</th>
202                             <th style={{ width: '14%' }}>Product Code <span className="required-marker">*</
span></th>
203                             <th style={{ width: '18%' }}>Product Name</th>
204                             <th style={{ width: '6%' }} className="text-center">Unit</th>
205                             <th style={{ width: '8%' }} className="text-right">Qty <span
className="required-marker">*</span></th>
206                             <th style={{ width: '10%' }} className="text-right">Unit Price <span
className="required-marker">*</span></th>
207                             <th style={{ width: '10%' }} className="text-right">Extended</th>
208                             <th style={{ width: '8%' }} className="text-right">Disc %</th>
209                             <th style={{ width: '10%' }} className="text-right">Disc Amt</th>
210                             <th style={{ width: '10%' }} className="text-right">Net Price</th>
211                             <th style={{ width: '100px' }} className="text-center">Actions</th>
212                         </tr>
213                     </thead>
214                     <tbody>
215                         {items.map((it, i) => (
216                             <LineItemRow
217                                 key={i}
218                                 index={i}
219                                 item={it}
220                                 itemsLength={items.length}
221                                 update={update}
222                                 removeRow={removeRow}
223                                 moveUp={moveUp}
224                                 moveDown={moveDown}
225                                 insertRowAfter={insertRowAfter}
226                                 isDragging={dragIndex === i}
227                                 isDragOver={dragOverIndex === i && dragIndex !== i}

```

```

228         onDragStart={handleDragStart}
229         onDragOver={handleDragOver}
230         onDragLeave={handleDragLeave}
231         onDrop={handleDrop}
232         onDragEnd={handleDragEnd}
233         productErrorRow={productErrorRow}
234         setProductErrorRow={setProductErrorRow}
235         onPickProduct={onPickProduct}
236         onOpenPicker={({idx} => { setPickerRowIndex(idx);
setProductPickerOpen(true); }}
237         onProductCodeBlur={handleProductCodeBlur}
238         formatBaht={formatBaht}
239         computeExtended={computeExtended}
240         computeDiscountAmount={computeDiscountAmount}
241         computeNetPrice={computeNetPrice}
242     />
243   )))
244   {items.length === 0 && (
245     <tr>
246       <td colSpan="11" style={{
247         padding: 40,
248         textAlign: 'center',
249         color: 'var(--text-muted)'
250       }}>
251         <svg style={{ marginBottom: 12, opacity: 0.5 }} width="48" height="48"
viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.5" strokeLinecap="round"
strokeLinejoin="round">
252           <path d="M21 16V8a2 2 0 0 1-1.73l-7-4a2 2 0 0 2 0l-7 4A2 2 0 0
0 3 8v8a2 2 0 0 1 1.73l7 4a2 2 0 0 2 0l7-4A2 2 0 0 21 16z"></path>
253         </svg>
254         <div>No items added yet</div>
255         <div style={{ fontSize: '0.85rem', marginTop: 4 }}>Click "Add Item" to
start</div>
256       </td>
257     </tr>
258   </tbody>
259 </table>
260 </div>
261
262   { /* Subtotal Footer */
263   {items.length > 0 && (
264     <div style={{
265       display: 'flex',
266       justifyContent: 'flex-end',
267       marginTop: 16,
268       padding: 16,
269       borderTop: '2px solid var(--border)'
270     }}>
271     <div style={{
272       display: 'flex',
273       alignItems: 'center',
274       gap: 32,
275       padding: '12px 20px',
276       background: 'var(--bg-body)',
277       borderRadius: 'var(--radius-sm)'
278     }}>
279       <div style={{ display: 'flex', flexDirection: 'column', alignItems: 'flex-end',
gap: 4 }}>
280         <span style={{ fontSize: '0.85rem', color: 'var(--text-muted)' }}>
281           Total Price ({items.length} item{items.length !== 1 ? 's' : ''})
282         </span>
283         <span style={{ fontSize: '1.1rem', fontWeight: 700, color: 'var(--primary)' }}>
284           {formatBaht(totalPrice)}
285         </span>
286       </div>
287       {totalDiscount > 0 && (
288         <div style={{ display: 'flex', flexDirection: 'column', alignItems: 'flex-
end', gap: 4 }}>
289           <span style={{ fontSize: '0.85rem', color: 'var(--text-muted)' }}>Total
Discount</span>
290           <span style={{ fontSize: '1.1rem', fontWeight: 700, color: '#ef4444' }}>
291             -{formatBaht(totalDiscount)}
292           </span>
293         </div>
294       </div>
295     </div>
296   </div>
297 </div>
298   )}

```

```

299
300     <ProductPickerModal
301         isOpen={productPickerOpen}
302         onClose={() => setProductPickerOpen(false)}
303         initialSearch={items[pickerRowIndex]?.product_code ?? ""}
304         onSelect={(row) => {
305             const productData = {
306                 code: row.code,
307                 name: row.name,
308                 label: `${row.code} - ${row.name}`,
309                 units_code: row.units_code,
310                 unit_price: Number(row.unit_price || 0),
311             };
312             onPickProduct(pickerRowIndex, row.code, productData);
313         }}
314     />
315 </div>
316 );
317 }
318

```

Line Item Row (Disc %, Disc Amt, Net Price)

```

1 // Single row in the line items table: product code, name, qty, unit price, discount, reorder/remove
  buttons
2 import React from "react";
3 import { formatBaht } from "../utils.js";
4
5 const actionBtnStyle = {
6   background: "none",
7   border: "none",
8   cursor: "pointer",
9   padding: 4,
10  borderRadius: 4,
11  color: "#666",
12  transition: "all 0.15s",
13  display: "flex",
14  alignItems: "center",
15  justifyContent: "center",
16 };
17 const disabledBtnStyle = { ...actionBtnStyle, cursor: "not-allowed", color: "#ddd" };
18
19 const DragIcon = () => (
20   <svg width="16" height="16" viewBox="0 0 24 24" fill="currentColor">
21     <circle cx="9" cy="6" r="1.5" />
22     <circle cx="15" cy="6" r="1.5" />
23     <circle cx="9" cy="12" r="1.5" />
24     <circle cx="15" cy="12" r="1.5" />
25     <circle cx="9" cy="18" r="1.5" />
26     <circle cx="15" cy="18" r="1.5" />
27   </svg>
28 );
29 const ArrowUpIcon = () => (
30   <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2.5"
strokeLinecap="round" strokeLinejoin="round">
31     <polyline points="18 15 12 9 6 15"></polyline>
32   </svg>
33 );
34 const ArrowDownIcon = () => (
35   <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2.5"
strokeLinecap="round" strokeLinejoin="round">
36     <polyline points="6 9 12 15 18 9"></polyline>
37   </svg>
38 );
39
40 export default function LineItemRow({
41   index,
42   item,
43   itemsLength,
44   update,
45   removeRow,
46   moveUp,
47   moveDown,
48   insertRowAfter,
49   isDragging,
50   isDragOver,
51   onDragStart,
52   onDragOver,
53   onDragLeave,
54   onDrop,
55   onDragEnd,
56   productErrorRow,
57   setProductErrorRow,
58   onPickProduct,
59   onOpenPicker,
60   onProductCodeBlur,
61   formatBaht,
62   computeExtended,
63   computeDiscountAmount,
64   computeNetPrice,
65 }) {
66   const i = index;
67   const it = item;
68   return (
69     <tr
70       draggable
71       onDragStart={(e) => onDragStart(e, i)}
72       onDragOver={(e) => onDragOver(e, i)}
73       onDragLeave={onDragLeave}
74       onDrop={(e) => onDrop(e, i)}

```

```

75     onDragEnd={onDragEnd}
76     style={{
77         opacity: isDragging ? 0.5 : 1,
78         background: isDragOver ? "var(--primary-light, #e0f2fe)" : "transparent",
79         transition: "background 0.15s, opacity 0.15s",
80         cursor: "grab",
81     }}
82     >
83     <td className="text-center">
84         <div style={{ display: "flex", alignItems: "center", justifyContent: "center", gap: 4, color:
"var(--text-muted)" }}>
85             <span style={{ cursor: "grab", color: "#aaa" }} title="Drag to reorder">
86                 <DragIcon />
87             </span>
88             <span style={{ fontWeight: 600, fontSize: "0.85rem", color: "var(--text-muted)", minWidth: 20 }}>
>{i + 1}</span>
89         </div>
90     </td>
91     <td>
92         <div>
93             <div style={{ display: "flex", gap: 6, alignItems: "stretch" }}>
94                 <input
95                     type="text"
96                     className="form-control"
97                     value={it.product_code || ""}
98                     onChange={(e) => {
99                         update(i, { product_code: e.target.value });
100                         if (productErrorRow === i) setProductErrorRow(null);
101                     }}
102                     onBlur={() => onProductCodeBlur(i)}
103                     placeholder="e.g. P001"
104                     style={{ flex: 1, minWidth: 0, padding: "6px 10px", fontSize: "0.9rem", borderColor:
productErrorRow === i ? "#ef4444" : undefined }}
105                 </>
106                 <button
107                     type="button"
108                     className="btn btn-primary"
109                     style={{ flexShrink: 0, padding: "6px 12px", fontSize: "0.8rem" }}
110                     onClick={() => onOpenPicker(i)}
111                     title="List of Values"
112                 >
113                     LoV
114                 </button>
115                 {it.product_code && (
116                     <button
117                         type="button"
118                         onClick={(e) => {
119                             e.stopPropagation();
120                             onPickProduct(i, "", null);
121                             setProductErrorRow(null);
122                         }}
123                         title="Clear"
124                         style={{
125                             padding: "0 8px",
126                             border: "1px solid var(--border)",
127                             borderRadius: "var(--radius-sm)",
128                             background: "var(--bg-body)",
129                             color: "var(--text-muted)",
130                             cursor: "pointer",
131                             fontSize: "1.1rem",
132                             lineHeight: 1,
133                         }}
134                     >
135                         x
136                     </button>
137                 )}
138             </div>
139             {productErrorRow === i && (
140                 <span style={{ fontSize: "0.75rem", color: "#ef4444", marginTop: 4, display: "block" }}>
>Product not found</span>
141             )}
142         </div>
143     </td>
144     <td>
145         <input
146             type="text"
147             className="form-control"
148             disabled
149             readOnly

```



```

150     value={it.product_name ?? it.product_label?.replace(/^[^\-]+\s*-\s*/, "") ?? ""}
151     placeholder="-"
152     style={{ padding: "6px 10px", fontSize: "0.9rem", background: "var(--bg-body)" }}
153   />
154 </td>
155 <td className="text-center">
156   <span
157     style={{
158       display: "inline-block",
159       padding: "4px 10px",
160       background: "var(--bg-body)",
161       borderRadius: 4,
162       fontSize: "0.8rem",
163       color: "var(--text-muted)",
164       fontWeight: 500,
165     }}
166   >
167     {it.units_code || "-"}
168   </span>
169 </td>
170 <td>
171   <input
172     type="number"
173     step="0.01"
174     min="0.01"
175     value={it.quantity}
176     onChange={(e) => update(i, { quantity: e.target.value })}
177     className="form-control"
178     style={{ textAlign: "right", padding: "8px 12px", fontSize: "0.9rem" }}
179   />
180 </td>
181 <td>
182   <div
183     style={{
184       textAlign: "right",
185       padding: "8px 12px",
186       background: "var(--bg-body)",
187       borderRadius: "var(--radius-sm)",
188       color: "var(--text-muted)",
189       fontSize: "0.9rem",
190     }}
191   >
192     {formatBaht(it.unit_price)}
193   </div>
194 </td>
195 <td>
196   <div style={{ textAlign: "right", fontWeight: 600, color: "var(--primary)", fontSize: "0.95rem" }}>
197     {formatBaht(computeExtended(it))}
198   </div>
199 </td>
200 <td>
201   <input
202     type="number"
203     step="0.01"
204     min="0"
205     max="100"
206     value={it.line_discount_percent ?? 0}
207     onChange={(e) => update(i, { line_discount_percent: e.target.value })}
208     className="form-control"
209     style={{ textAlign: "right", padding: "8px 12px", fontSize: "0.9rem" }}
210   />
211 </td>
212 <td>
213   <div style={{ textAlign: "right", color: "#ef4444", fontSize: "0.9rem" }}>
214     {computeDiscountAmount(it) > 0 ? formatBaht(computeDiscountAmount(it)) : "-"}
215   </div>
216 </td>
217 <td>
218   <div style={{ textAlign: "right", fontWeight: 600, color: "var(--text-main)", fontSize:
"0.95rem" }}>
219     {formatBaht(computeNetPrice(it))}
220   </div>
221 </td>
222 <td>
223   <div style={{ display: "flex", alignItems: "center", justifyContent: "center", gap: 2 }}>
224     <button type="button" onClick={() => moveUp(i)} disabled={i === 0} style={i === 0 ?
disabledBtnStyle : actionBtnStyle} title="Move up">
225       <ArrowUpIcon />
226     </button>

```

```

227     <button
228         type="button"
229         onClick={() => moveDown(i)}
230         disabled={i === itemsLength - 1}
231         style={i === itemsLength - 1 ? disabledBtnStyle : actionBtnStyle}
232         title="Move down"
233     >
234         <ArrowDownIcon />
235     </button>
236     <button type="button" onClick={() => insertRowAfter(i)} style={actionBtnStyle} title="Insert row
below">
237         <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor"
strokeWidth="2.5" strokeLinecap="round" strokeLinejoin="round">
238             <line x1="12" y1="5" x2="12" y2="19"></line>
239             <line x1="5" y1="12" x2="19" y2="12"></line>
240         </svg>
241     </button>
242     <button
243         type="button"
244         onClick={() => removeRow(i)}
245         disabled={itemsLength <= 1}
246         style={{
247             ...actionBtnStyle,
248             color: itemsLength <= 1 ? "#ddd" : "#ef4444",
249             cursor: itemsLength <= 1 ? "not-allowed" : "pointer",
250         }}
251         title="Remove item"
252     >
253         <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor"
strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
254             <polyline points="3 6 5 6 21 6"></polyline>
255             <path d="M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2 2V6m3 0V4a2 2 0 0 1 2-2h4a2 2 0 0 1 2 2v2"></
path>
256         </svg>
257     </button>
258 </div>
259 </td>
260 </tr>
261 );
262 }
263

```

Invoice Form (vatPercent, config API, summary)

```

1 // Invoice form (create/edit) with customer select and line items
2 // Example usage: <InvoiceForm onSubmit={...} />
3 import React from "react";
4 import LineItemsEditor from "../LineItemsEditor.jsx";
5 import CustomerPickerModal from "../CustomerPickerModal.jsx";
6 import SalesPersonPickerModal from "../SalesPersonPickerModal.jsx";
7 import { AlertModal } from "../Modal.jsx";
8 import { getCustomer } from "../api/customers.api.js";
9 import { getConfig } from "../api/configuration.api.js";
10 import { formatBaht } from "../utils.js";
11
12 export default function InvoiceForm({ onSubmit, submitting, initialData }) {
13   // Local state for header fields and line items
14   const [invoiceNo, setInvoiceNo] = React.useState("");
15   const [customerCode, setCustomerCode] = React.useState("");
16   const [invoiceDate, setInvoiceDate] = React.useState(new Date().toISOString().slice(0, 10));
17   const [vatPercent, setVatPercent] = React.useState(7);
18   const [items, setItems] = React.useState([{ product_code: "", quantity: 1, unit_price: 0,
line_discount_percent: 0 }]);
19   const [alertModal, setAlertModal] = React.useState({ isOpen: false, title: "Validation Error", message:
"" });
20   const [customerModalOpen, setCustomerModalOpen] = React.useState(false);
21   const [customerDetails, setCustomerDetails] = React.useState(null);
22   const [customerLoadError, setCustomerLoadError] = React.useState("");
23   const [salesPersonCode, setSalesPersonCode] = React.useState("");
24   const [salesPersonName, setSalesPersonName] = React.useState("");
25   const [salesPersonModalOpen, setSalesPersonModalOpen] = React.useState(false);
26
27   // Fetch vat_percent from config on create mode
28   React.useEffect(() => {
29     if (!initialData) {
30       getConfig("vat_percent")
31         .then((val) => setVatPercent(Number(val)))
32         .catch(() => {});
33     }
34   }, []);
35
36   // When customer code is set (from LoV or initialData), fetch name and address
37   React.useEffect(() => {
38     const code = String(customerCode || "").trim();
39     if (!code) {
40       setCustomerDetails(null);
41       setCustomerLoadError("");
42       return;
43     }
44     setCustomerLoadError("");
45     let cancelled = false;
46     getCustomer(code)
47       .then((data) => {
48         if (!cancelled) setCustomerDetails(data);
49       })
50       .catch(() => {
51         if (!cancelled) {
52           setCustomerDetails(null);
53           setCustomerLoadError("Customer not found");
54         }
55       });
56     return () => { cancelled = true; };
57   }, [customerCode]);
58
59   // Load customer by code on blur (user typed code)
60   const handleCustomerCodeBlur = () => {
61     const code = String(customerCode || "").trim();
62     if (!code) {
63       setCustomerDetails(null);
64       setCustomerLoadError("");
65       return;
66     }
67     setCustomerLoadError("");
68     getCustomer(code)
69       .then((data) => setCustomerDetails(data))
70       .catch(() => {
71         setCustomerDetails(null);
72         setCustomerLoadError("Customer not found");
73       });
74   };
75

```

```

76   React.useEffect(() => {
77     if (initialData) {
78       setInvoiceNo(initialData.invoice_no);
79       setCustomerCode(initialData.customer_code || "");
80       setSalesPersonCode(initialData.sales_person_code || "");
81       setSalesPersonName(initialData.sales_person_name || "");
82       const d = initialData.invoice_date ? new Date(initialData.invoice_date).toISOString().slice(0, 10) :
"";
83       setInvoiceDate(d);
84       setVatPercent(Math.round(Number(initialData.vat_rate || 0.07) * 100));
85       const mappedItems = (initialData.line_items || []).map(li => ({
86         line_item_id: li.line_item_id,
87         product_code: li.product_code || "",
88         product_label: li.product_label || `${li.product_code || ""} - ${li.product_name || ""}`.replace(/
^ - /, ""),
89         units_code: li.units_code || "",
90         quantity: li.quantity,
91         unit_price: Number(li.unit_price),
92         line_discount_percent: Number(li.line_discount_percent || 0),
93       }));
94       setItems(mappedItems.length > 0 ? mappedItems : [{ product_code: "", quantity: 1, unit_price: 0,
line_discount_percent: 0 }]);
95     }
96   }, [initialData]);
97
98   const vatRate = Number(vatPercent) / 100;
99   const totalPrice = items.reduce((s, it) => s + Number(it.quantity || 0) * Number(it.unit_price || 0), 0);
100  const totalDiscount = items.reduce((s, it) => {
101    const extended = Number(it.quantity || 0) * Number(it.unit_price || 0);
102    const pct = Number(it.line_discount_percent || 0);
103    return s + Math.round(extended * pct / 100 * 100) / 100;
104  }, 0);
105  const netPrice = totalPrice - totalDiscount;
106  const vatAmount = Math.round(netPrice * vatRate * 100) / 100;
107  const amountDue = netPrice + vatAmount;
108
109  const [autoCode, setAutoCode] = React.useState(true);
110
111  const hasEmptyProduct = items.some(it => !it.product_code);
112  const hasEmptyCustomer = !String(customerCode || "").trim() || !customerDetails;
113
114  const customerAddressDisplay = customerDetails
115    ? [customerDetails.address_line1, customerDetails.address_line2,
customerDetails.country_name].filter(Boolean).join(", ") || ""
116    : "";
117
118  // Collect all validation errors; return empty array if valid.
119  function validate() {
120    const errs = [];
121    if (!invoiceDate || String(invoiceDate).trim() === "") errs.push("Date should not be null");
122    if (!String(customerCode || "").trim()) errs.push("Customer Code should not be null");
123    else if (!customerDetails) errs.push("Customer must be selected from list (enter code and blur, or use
LoV)");
124    if (!initialData && !autoCode && !String(invoiceNo || "").trim()) errs.push("Invoice No should not be
null");
125    items.forEach((it, i) => {
126      const row = i + 1;
127      if (!String(it.product_code || "").trim()) errs.push(`Row ${row}: Product is required`);
128      else {
129        const q = Number(it.quantity);
130        if (Number.isNaN(q) || q <= 0) errs.push(`Row ${row}: Product quantity should be a positive
number`);
131        const p = Number(it.unit_price);
132        if (it.unit_price !== "" && it.unit_price !== null && (Number.isNaN(p) || p < 0)) errs.push(`Row
${row}: Price should be a positive number`);
133      }
134    });
135    return errs;
136  }
137
138  // Build payload and pass to parent
139  function handleSubmit(e) {
140    e.preventDefault();
141    const errors = validate();
142    if (errors.length > 0) {
143      showAlertModal({
144        isOpen: true,
145        title: "Save Failed.",
146        message: (

```

```

147         <ul style={{ margin: 0, paddingLeft: 20, color: "var(--text-main)" }}>
148             {errors.map((msg, i) => (
149                 <li key={i} style={{ marginBottom: 4 }}>{msg}</li>
150             ))}
151         </ul>
152     ),
153 });
154     return;
155 }
156
157 const payload = {
158     invoice_no: initialData ? invoiceNo.trim() : (autoCode ? "" : invoiceNo.trim()),
159     customer_code: String(customerCode).trim(),
160     sales_person_code: String(salesPersonCode || "").trim() || undefined,
161     invoice_date: invoiceDate,
162     vat_rate: Number(vatPercent) / 100,
163     line_items: items.map((x) => {
164         const out = {
165             product_code: String(x.product_code || "").trim(),
166             quantity: Number(x.quantity),
167             unit_price: x.unit_price === "" || x.unit_price === null ? undefined : Number(x.unit_price),
168             line_discount_percent: Number(x.line_discount_percent || 0),
169         };
170         if (x.line_item_id != null && Number(x.line_item_id) > 0) out.id = Number(x.line_item_id);
171         return out;
172     }),
173 };
174     onSubmit(payload);
175 }
176
177 return (
178     <>
179         <AlertModal
180             isOpen={alertModal.isOpen}
181             onClose={() => setAlertModal((prev) => ({ ...prev, isOpen: false })))}
182             title={alertModal.title}
183             message={alertModal.message}
184         />
185         <form onSubmit={handleSubmit} className="invoice-form">
186             <div className="invoice-form-grid" style={{ display: "grid", gridTemplateColumns: "1fr 280px", gap:
16, marginBottom: 16 }}>
187                 <div className="card">
188                     <h4>Invoice Details</h4>
189                     <div style={{ display: "grid", gap: 12 }}>
190                         <div className="form-group">
191                             <label className="form-label">{(!initialData && autoCode) ? "Invoice No" : <>Invoice No
192 <span className="required-marker">*</span></></label>
193                             <div className="flex gap-2">
194                                 <input
195                                     className="form-control"
196                                     required={!autoCode}
197                                     disabled={autoCode}
198                                     value={invoiceNo}
199                                     onChange={(e) => setInvoiceNo(e.target.value)}
200                                     placeholder="e.g. INV-2026-0001"
201                                 />
202                                 {!initialData && (
203                                     <div className="form-inline-option">
204                                         <input type="checkbox" checked={autoCode} onChange={e =>
205 setAutoCode(e.target.checked)} id="inv_auto" />
206                                         <label htmlFor="inv_auto">Auto</label>
207                                     </div>
208                                 )}
209                             </div>
210                         </div>
211                         <div className="form-group">
212                             <label className="form-label">Customer Code <span className="required-marker">*</span></
213 label>
214                             <div style={{ display: "flex", gap: 8, alignItems: "stretch" }}>
215                                 <input
216                                     className="form-control"
217                                     value={customerCode}
218                                     onChange={(e) => setCustomerCode(e.target.value)}
219                                     onBlur={handleCustomerCodeBlur}
220                                     placeholder="e.g. C001"
221                                     style={{ flex: 1 }}
222                                 />
223                                 <button

```

```

222         type="button"
223         className="btn btn-primary"
224         onClick={() => setCustomerModalOpen(true)}
225         title="List of Values"
226     >
227         LoV
228     </button>
229     {customerCode && (
230     <button
231         type="button"
232         onClick={() => { setCustomerCode(""); setCustomerDetails(null);
setCustomerLoadError(""); }}
233         title="Clear"
234         style={{
235             padding: "0 12px",
236             border: "1px solid var(--border)",
237             borderRadius: "var(--radius-sm)",
238             background: "var(--bg-body)",
239             color: "var(--text-muted)",
240             cursor: "pointer",
241             fontSize: "1.2rem",
242             lineHeight: 1,
243         }}
244     >
245         x
246     </button>
247     )}
248 </div>
249     {customerLoadError && (
250     <span style={{ fontSize: "0.8rem", color: "#ef4444", marginTop: 4, display: "block" }}
>{customerLoadError}</span>
251     )}
252     {!customerCode && (
253     <span style={{ fontSize: "0.8rem", color: "var(--text-muted)", marginTop: 4, display:
"block" }}>Required. Type code and blur, or use LoV to select.</span>
254     )}
255 </div>
256
257 <div className="form-group">
258     <label className="form-label">Customer Name</label>
259     <input className="form-control" disabled value={customerDetails?.name ?? ""} readOnly
placeholder="-" />
260 </div>
261
262 <div className="form-group">
263     <label className="form-label">Customer Address</label>
264     <input className="form-control" disabled value={customerAddressDisplay} readOnly
placeholder="-" />
265 </div>
266
267 {/* Sales Person Code: input + LoV button + clear */}
268 <div className="form-group">
269     <label className="form-label">Sales Person Code</label>
270     <div style={{ display: "flex", gap: 8 }}>
271         <input
272             className="form-control"
273             value={salesPersonCode}
274             onChange={(e) => {
275                 setSalesPersonCode(e.target.value);
276                 setSalesPersonName("");
277             }}
278             placeholder="e.g. SP001"
279             style={{ flex: 1 }}
280         />
281         <button type="button" className="btn btn-primary"
282             onClick={() => setSalesPersonModalOpen(true)}>
283             LoV
284         </button>
285         {salesPersonCode && (
286         <button type="button" className="btn btn-secondary"
287             onClick={() => {
288                 setSalesPersonCode("");
289                 setSalesPersonName("");
290             }}
291             style={{
292                 padding: "0 12px",
293                 border: "1px solid var(--border)",
294                 borderRadius: "var(--radius-sm)",
295                 background: "var(--bg-body)",

```

```

296         color: "var(--text-muted)",
297         cursor: "pointer",
298         fontSize: "1.2rem",
299         lineHeight: 1,
300     }>
301     x
302 </button>
303 }
304 </div>
305 </div>
306
307 <div className="form-group">
308   <label className="form-label">Sales Person Name</label>
309   <input className="form-control" disabled value={salesPersonName} placeholder="-" />
310 </div>
311
312 <CustomerPickerModal
313   isOpen={customerModalOpen}
314   onClose={() => setCustomerModalOpen(false)}
315   initialSearch={customerCode}
316   onSelect={(code) => {
317     setCustomerCode(String(code));
318     setCustomerModalOpen(false);
319   }}
320 />
321
322 <SalesPersonPickerModal
323   isOpen={salesPersonModalOpen}
324   onClose={() => setSalesPersonModalOpen(false)}
325   onSelect={(code, name) => {
326     setSalesPersonCode(code);
327     setSalesPersonName(name);
328     setSalesPersonModalOpen(false);
329   }}
330 />
331
332 <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 12 }}>
333   <div className="form-group">
334     <label className="form-label">Invoice Date <span className="required-marker">*</span></label>
335     <input
336       type="date"
337       className="form-control"
338       value={invoiceDate}
339       onChange={(e) => setInvoiceDate(e.target.value)}
340     />
341   </div>
342   <div className="form-group">
343     <label className="form-label">VAT Rate</label>
344     <div style={{ display: 'flex', alignItems: 'center', gap: 8 }}>
345       <input
346         type="number"
347         step="1"
348         min="0"
349         max="100"
350         className="form-control"
351         value={vatPercent}
352         onChange={(e) => setVatPercent(e.target.value)}
353       />
354       <span style={{ fontSize: '0.8rem', color: 'var(--text-muted)', whiteSpace: 'nowrap' }}>
355         %</span>
356     </div>
357   </div>
358 </div>
359 </div>
360
361 <div className="card invoice-summary-card" style={{ height: "fit-content" }}>
362   <h4>Summary</h4>
363   <div style={{ display: "grid", gap: 8 }}>
364     <div style={{ display: "flex", justifyContent: "space-between", fontSize: "0.875rem", color:
365       "var(--text-muted)" }}>
366       <span>Total Price</span>
367       <span className="amount">{submitting ? "...": formatBaht(totalPrice)}</span>
368     </div>
369     <div style={{ display: "flex", justifyContent: "space-between", fontSize: "0.875rem", color:
370       "var(--text-muted)" }}>
371       <span>Total Discount</span>
372       <span className="amount">{submitting ? "...": formatBaht(totalDiscount)}</span>

```

```

371         </div>
372         <div style={{ display: "flex", justifyContent: "space-between", fontSize: "0.875rem", color:
"var(--text-muted)" }}>
373             <span>Net Price</span>
374             <span className="amount">{submitting ? "...": formatBaht(netPrice)}</span>
375         </div>
376         <div style={{ display: "flex", justifyContent: "space-between", fontSize: "0.875rem", color:
"var(--text-muted)" }}>
377             <span>VAT ({Number(vatPercent).toFixed(0)}%)</span>
378             <span className="amount">{submitting ? "...": formatBaht(vatAmount)}</span>
379         </div>
380         <div style={{ borderTop: "1px solid var(--border)", padding: 10, margin: 2, display:
"flex", justifyContent: "space-between", fontSize: "1.1rem", fontWeight: 700, color: "var(--primary)" }}>
381             <span>Amount Due</span>
382             <span>{submitting ? "...": formatBaht(amountDue)}</span>
383         </div>
384     </div>
385     <div style={{ marginTop: 16 }}>
386         <button
387             type="submit"
388             className="btn btn-primary"
389             style={{ width: "100%", padding: "10px 14px", fontSize: "0.875rem" }}
390             disabled={submitting || hasEmptyProduct || hasEmptyCustomer}
391             >
392             {submitting ? (initialData ? "Saving..." : "Creating...") : (initialData ? "Save Changes" :
"Create Invoice")}
393         </button>
394         {(hasEmptyCustomer || hasEmptyProduct) && (
395             <div style={{ marginTop: 6, fontSize: '0.75rem', color: '#ef4444', textAlign: 'center' }}>
396                 {hasEmptyCustomer && <div>Please enter customer code or select from LoV</div>}
397                 {hasEmptyProduct && <div>Please select a product for all items</div>}
398             </div>
399         )}
400     </div>
401 </div>
402 </div>
403
404     <LineItemsEditor value={items} onChange={setItems} />
405 </form>
406 </>
407 );
408 }
409

```


Invoice Page (view/edit discount columns)

```

1 // Invoice page (View, Create & Edit)
2 // - /invoices/new !' mode="create"
3 // - /invoices/:id !' mode="view"
4 // - /invoices/:id/edit !' mode="edit"
5 import React from "react";
6 import { useNavigate, Link, useParams } from "react-router-dom";
7 import { getInvoice, createInvoice, updateInvoice } from "../../api/invoices.api.js";
8 import { toast } from "react-toastify";
9 import { formatBaht, formatDate } from "../../utils.js";
10 import InvoiceForm from "../../components/InvoiceForm.jsx";
11 import Loading from "../../components/Loading.jsx";
12
13 export default function InvoicePage({ mode: propMode }) {
14   const { id } = useParams();
15   const mode = propMode || (id ? "view" : "create");
16   const nav = useNavigate();
17
18   const [invoiceData, setInvoiceData] = React.useState(null);
19   const [initialData, setInitialData] = React.useState(null);
20   const [err, setErr] = React.useState("");
21   const [submitting, setSubmitting] = React.useState(false);
22   const [loading, setLoading] = React.useState(true);
23
24   React.useEffect(() => {
25     if (mode === "create") {
26       setLoading(false);
27     } else if (mode === "view") {
28       getInvoice(id)
29         .then((data) => {
30           setInvoiceData(data);
31           setLoading(false);
32         })
33         .catch((e) => {
34           setErr(String(e.message || e));
35           setLoading(false);
36         });
37     } else {
38       // edit mode
39       getInvoice(id)
40         .then((inv) => {
41           setInvoiceData(inv);
42
43           const h = inv.header;
44           setInitialData({
45             invoice_no: h.invoice_no,
46             customer_code: h.customer_code,
47             customer_label: `${h.customer_code} - ${h.customer_name}`.replace(/^- /,
48             ' '),
49             sales_person_code: h.sales_person_code || "",
50             sales_person_name: h.sales_person_name || "",
51             invoice_date: h.invoice_date,
52             vat_rate: h.vat_rate !== null ? Number(h.vat_rate) : 0.07,
53             line_items: inv.line_items.map(li => ({
54               line_item_id: li.id,
55               product_code: li.product_code,
56               product_name: li.product_name,
57               product_label: `${li.product_code} - ${li.product_name}`,
58               units_code: li.units_code,
59               quantity: li.quantity,
60               unit_price: li.unit_price,
61               line_discount_percent: Number(li.line_discount_percent || 0),
62             })))
63           });
64           setLoading(false);
65         })
66         .catch((e) => {
67           setErr(String(e.message || e));
68           setLoading(false);
69         });
70     }
71   }, [id, mode]);
72
73   async function onSubmit(payload) {
74     setErr("");
75     setSubmitting(true);
76     try {
77       if (mode === "create") {

```

```

77         const res = await createInvoice(payload);
78         toast.success("Invoice created.");
79         nav(`/invoices/${encodeURIComponent(res.invoice_no)}`);
80     } else {
81         await updateInvoice(id, payload);
82         toast.success("Invoice updated.");
83         nav(`/invoices/${encodeURIComponent(id)}`);
84     }
85 } catch (e) {
86     const msg = String(e.message || e);
87     setErr(msg);
88     toast.error(msg);
89 } finally {
90     setSubmitting(false);
91 }
92 }
93
94 const handlePrint = () => window.print();
95
96 if (loading) return <Loading size="large" />;
97
98 const isView = mode === "view";
99 const isCreate = mode === "create";
100
101 // View Mode - Invoice Preview with Print
102 if (isView && invoiceData) {
103     const h = invoiceData.header;
104     const lines = invoiceData.line_items || [];
105     const vatRate = Number(h.vat_rate || 0.07);
106     const totalPrice = Number(h.total_price || h.total_amount || 0);
107     const totalDiscount = Number(h.total_discount || 0);
108     const netPrice = totalPrice - totalDiscount;
109     const vatAmount = Number(h.vat_amount || h.vat || 0);
110
111     return (
112         <div className="invoice-preview">
113             <div className="page-header no-print">
114                 <h3 className="page-title">Invoice {h.invoice_no}</h3>
115                 <div className="flex gap-4">
116                     <Link to="/invoices" className="btn btn-outline">• Back</Link>
117                     <Link to={`/invoices/${id}/edit`} className="btn btn-outline">Edit</Link>
118                     <button onClick={handlePrint} className="btn btn-primary">
119                         <svg style={{ marginRight: 8 }} width="16" height="16" viewBox="0 0 24 24"
120                         fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"><path d="M6
121                         9V2h12v7"></path><path d="M6 18H4a2 2 0 0 1-2-2v-5a2 2 0 0 1 2-2h16a2 2 0 0 1 2 2v5a2 2 0 0 1-2 2h-2"></
122                         path><rect x="6" y="14" width="12" height="8"></rect></svg>
123                         Print PDF
124                     </button>
125                 </div>
126             </div>
127             <div>
128                 <div className="card">
129                     <div className="flex justify-between mb-4">
130                         <div>
131                             <div className="brand mb-4">InvoiceDoc v2</div>
132                             <div className="font-bold">Customer</div>
133                             <div>{h.customer_name}</div>
134                             <div className="text-muted">{h.address_line1 || "-"}</div>
135                             <div className="text-muted">{h.address_line2 || ""}</div>
136                             <div className="text-muted">{h.country_name || "-"}</div>
137                         </div>
138                         <div className="text-right">
139                             <h2 className="mb-4">INVOICE</h2>
140                             <div><span className="font-bold">Date:</span> {formatDate(h.invoice_date)}</div>
141
142                             <div><span className="font-bold">Invoice No:</span> {h.invoice_no}</div>
143                             {h.sales_person_code && (
144                                 <div>
145                                     <span className="font-bold">Sales Person:</span> {h.sales_person_code} -
146                                     {h.sales_person_name}
147                                 </div>
148                             )}
149                         </div>
150                     </div>
151                 </div>
152             </div>
153             <div className="table-container">
154                 <table className="modern-table">
155                     <thead>
156                         <tr>

```

```

151         <th>Product</th>
152         <th>Unit</th>
153         <th className="text-right">Qty</th>
154         <th className="text-right">Unit Price</th>
155         <th className="text-right">Extended</th>
156         <th className="text-right">Disc %</th>
157         <th className="text-right">Disc Amt</th>
158         <th className="text-right">Net Price</th>
159     </tr>
160 </thead>
161 <tbody>
162     {lines.map((li) => (
163         <tr key={li.id}>
164             <td>{li.product_code} - {li.product_name}</td>
165             <td>{li.units_code}</td>
166             <td className="text-right">{Number(li.quantity || 0).toFixed(2)}</
167             <td className="text-right">{formatBaht(li.unit_price)}</td>
168             <td className="text-right">{formatBaht(li.extended_price)}</td>
169             <td className="text-right">{Number(li.line_discount_percent ||
0).toFixed(2)}%</td>
170             <td className="text-right">{formatBaht(li.line_discount_amount)}</
171             <td className="text-right font-
bold">{formatBaht(li.line_net_price)}</td>
172         </tr>
173     )})
174 </tbody>
175 </table>
176 </div>
177
178 <div className="mt-4 flex justify-between">
179     <div className="no-print text-muted" style={{ maxWidth: 300, fontSize: '0.8rem' }}>
180         Thank you for your business. Please pay within 30 days.
181     </div>
182     <div style={{ minWidth: 240 }}>
183         <div className="flex justify-between mb-2">
184             <span>Total Price:</span>
185             <span>{formatBaht(totalPrice)}</span>
186         </div>
187         <div className="flex justify-between mb-2">
188             <span>Total Discount:</span>
189             <span>{formatBaht(totalDiscount)}</span>
190         </div>
191         <div className="flex justify-between mb-2">
192             <span>Net Price:</span>
193             <span>{formatBaht(netPrice)}</span>
194         </div>
195         <div className="flex justify-between mb-2">
196             <span>VAT ({(vatRate * 100).toFixed(0)}%)</span>
197             <span>{formatBaht(vatAmount)}</span>
198         </div>
199         <div className="flex justify-between mt-4 p-2 bg-body font-bold"
style={{ fontSize: '1.1rem' }}>
200             <span>Total Due:</span>
201             <span>{formatBaht(h.amount_due)}</span>
202         </div>
203     </div>
204 </div>
205 </div>
206 </div>
207 );
208 }
209
210 // Create/Edit Mode - Form
211 const title = isCreate ? "Create Invoice" : `Edit Invoice ${id}`;
212
213 return (
214     <div className="invoice-page">
215         <div className="page-header">
216             <h3 className="page-title">{title}</h3>
217             <Link to="/invoices" className="btn btn-outline">
218                 <svg style={{ marginRight: 8 }} width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"><line x1="19" y1="12" x2="5"
y2="12"></line><polyline points="12 19 5 12 12 5"></polyline></svg>
219                 Back
220             </Link>
221         </div>
222         {err && <div className="alert alert-error">{err}</div>}

```

```
223         <InvoiceForm
224             onSubmit={onSubmit}
225             submitting={submitting}
226             initialData={isCreate ? null : initialData}
227         />
228     </div>
229 );
230 }
231
```


